

1 Data Structures for Disjoint Sets

Disjoint sets (Union-Find)

Each set identified by a representative.

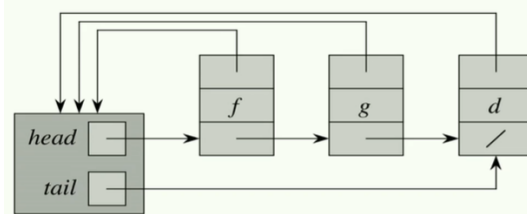
Operations

- **MAKE-SET(x)**: $S_i = \{x\}$, add to $\mathcal{S}$
Let $x \in S_x, y \in S_y$
- **UNION(x, y)**: $\mathcal{S} = \mathcal{S} \setminus \{S_x, S_y\} \cup \{S_x \cup S_y\}$
- **FIND(x)**: return representative of x in S_x

```

CONNECTED-COMPONENTS(G)
  for each vertex v in G.V
    MAKE-SET(v)
  for each edge (u, v) in G.E
    if FIND(u) != FIND(v)
      UNION(u, v)
    
```

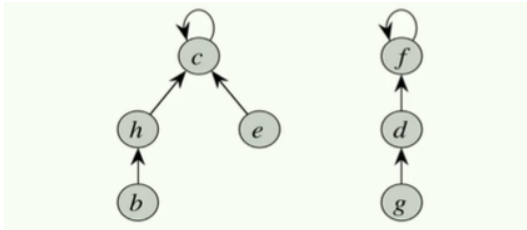
Linked list representation



Weighted-union heuristic

Always merge smaller into larger. m ops on n elements: $O(m + n \lg n)$. Each element's representative updated $\leq \lg n$ times (each update at least doubles set size).

Tree representation



Runtime with both heuristics

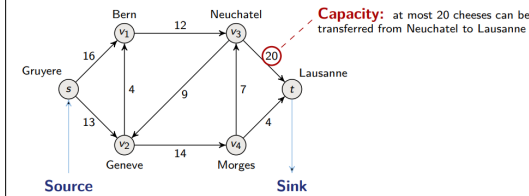
Union by rank: smaller-rank root $\rightarrow$ child of larger-rank root (rank $\approx$ height upper bound).

Path compression: all nodes on FIND path become direct children of root.

Union by rank + path compression: $O(m \cdot \alpha(n))$ for m operations on n elements. $\alpha(n) \leq 5$ for any practical n . Bound is tight.

2 Flow network

Definitions



Formally

- Source: $s \in V$, sink: $t \in V$
- Directed graph (DG): $G = (V, E)$
- Each (u, v) has a capacity $c(u, v) \geq 0$
- No antiparallel: if $(u, v) \in E$, then $(v, u) \notin E$

Capacity constraint

$$\forall u, v \in V, 0 \leq f(u, v) \leq c(u, v)$$

Flow conservation

$$\forall u \in V \setminus \{s, t\}, \sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$$

Cut

Partition of V into two disjoint sets S and T such that $s \in S$ and $t \in T$. $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$.

Maximum flow problem - Ford-Fulkerson method

Residual network $G_f = (V, E_f)$

$c_f(u, v)$: additional flow possible (forward edge) or removable (backward edge).

```

FORD-FULKERSON-METHOD(G, s, t)
  initialize flow f to 0
  while exists an augmenting path p in G_f
    b = bottleneck(p)
    augment flow along p by b
  return f
    
```

Min cut problem

Min cut

It is a cut where the capacity crossing it is the minimum among all cuts.

Net flow across cut

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$$

Max-flow min-cut theorem ($\Leftrightarrow$)

- f is the maximum flow
- G_f has no augmenting path
- $|f| = c(S, T)$ for a minimum cut (S, T)

Applications of max flow

Bipartite matching as flow

$G = (V, E)$, $V = L \cup R$. Connect $s \rightarrow \forall u \in L$ and $\forall v \in R \rightarrow t$, all capacities 1. Run Ford-Fulkerson.

Edge-disjoint paths

Max # edge-disjoint paths from s to $t = \min$ edges to remove to disconnect s from $t = \max$ flow (all capacities 1).

3 Spanning Trees

Definition

Set T of edges that is acyclic and spanning (connects all vertices of G).

Minimum Spanning Trees (MST) problem

In: undirected $G = (V, E)$, weights $w(u, v)$ for each $(u, v) \in E$.

Out: spanning tree of minimum total weight.

Cut property (safe edge)

Let $A \subseteq T_{\text{MST}}$, $(S, V \setminus S)$ a cut respecting A , and (u, v) a light (min-weight) edge crossing the cut. Then (u, v) is *safe* to add to A .

Prim's algorithm

```

PRIM(G, w, r)
  // w: E -> R weights
  Q = {} // min-priority queue (min-heap)
  for each u in G.V:
    u.key = inf
    u.pi = NIL // u.pi = parent in T
  INSERT(Q, u)
  DECREASE-KEY(Q, r, 0) // r.key = 0
  while Q != {}:
    u = EXTRACT-MIN(Q)
    for each v in G.Adj[u]:
      if v in Q and w(u, v) < v.key:
        v.pi = u
        DECREASE-KEY(Q, v, w(u, v))
    
```

Why does it work?

T is always a subtree of a MST.

Prim's runtime

$O(E \lg V)$ with binary heap. $O(V \lg V + E)$ with Fibonacci heap.

Integrals: $\int x^n \, dx = \frac{x^{n+1}}{n+1}, \int \frac{1}{x} \, dx = \ln |x|,$
 $\int e^x \, dx = e^x.$